

CSE499A – Section 15 – Group 03

Md. Rahad Hossain, Md. Yousuf, MD. Rokib Hasan Oli

CityPulse: Intelligent Parking & Traffic Control Platform

Weekly Progress Report – Update 2 (Sprint 2: System Design & Architecture)

I. SPRINT OVERVIEW

Sprint 2 covers system design and architecture. The goal is to finalize the full system design before writing feature code — service boundaries, data flow, database schema, and the initial skeleton for each application layer. Build tasks completed this sprint include the system architecture document, full PostgreSQL schema with migration, and scaffolded starting points for the backend, frontend, and mobile applications.

II. ARCHITECTURE DOCUMENT

A full system architecture document was produced covering ten areas: architectural style rationale, system layers, service boundaries, data flow diagrams for four core flows, messaging and event architecture, security and RBAC design, deployment topology, design patterns, and an ERD. The document is compiled as a PDF and stored at `applications/web/backend/docs/sysArch.pdf`.

A. Architectural Style

CityPulse uses a layered architecture with an event-driven core. The NestJS backend is structured as a modular monolith — clean module boundaries that can be extracted into separate services in CSE499B without a rewrite. FastAPI is the only hard service split, forced by the Python language requirement for AI/ML workloads.

B. Service Boundaries

NestJS owns all authentication, business logic, REST endpoints, WebSocket management, and geospatial queries. FastAPI owns RTSP ingestion, all model inference, and event publishing. They communicate via Redis Pub/Sub for async slot status events and internal HTTP for synchronous calls such as camera registration.

C. Messaging Architecture

Two message brokers are used with distinct reliability guarantees. Redis Pub/Sub handles real-time slot updates — a missed message is corrected by the next inference cycle in 2–3 seconds. RabbitMQ handles booking and payment events where delivery loss is not acceptable. This split is intentional and documented as an architecture decision record.

D. Permission-Based RBAC

Four roles are defined: admin, owner, authority, and driver. Authorization uses discrete permission strings stored in the database and embedded in JWT tokens at login. Route guards check the permissions array from the token directly — no per-request database lookup. Admin bypasses all permission checks. Adding access for a new role to a feature requires a single database row insertion, no code change.

III. DATABASE SCHEMA

A. PostgreSQL Schema

A complete Prisma schema was designed and applied to the local PostgreSQL database via a single migration (`20260708010310_init_schema`). The schema covers 17 tables across five domains.

RBAC: roles, permissions, role_permissions (many-to-many, seeded on init).

Users and Auth: users with phone and OAuth support; user_sessions replacing a simple refresh token table to support multiple concurrent sessions, admin force-revoke, and device tracking; user_oauth_accounts for Google, Facebook, and Apple sign-in.

Profiles: separate owner_profiles (business name, national ID, verification status), driver_profiles (FCM token, preferred payment method), and authority_profiles (organization, badge number, jurisdiction).

Parking: parking_spaces with Bangladesh administrative hierarchy (division, district, thana, area) for zone-based filtering; parking_slots with mask_polygon for CV patch cropping and confidence_score from AI inference; cameras with RTSP URL and heartbeat tracking; space_photos with S3 key references.

Transactions (future planned): bookings, payment_transactions (bKash, Nagad, card, cash with partial refund support), pricing_rules (surge threshold and multiplier in basis points).

Notifications: typed enum with FCM delivery tracking and JSON payload for deep link actions.

All tables include created_at, updated_at, and deleted_at for soft delete and audit. Indexes cover all foreign keys, status fields, and the Bangladesh zone hierarchy columns.

TABLE I
POSTGRESQL TABLES BY DOMAIN

Domain	Tables
RBAC	roles, permissions, role_permissions
Auth	users, user_sessions, user_oauth_accounts
Profiles	owner_profiles, driver_profiles, authority_profiles
Parking	parking_spaces, parking_slots, cameras, space_photos
Booking	bookings, payment_transactions, pricing_rules
Other	notifications

B. MongoDB Collections

MongoDB will store high-frequency append-only data produced by the AI service. Three collections are designed: `detection_events` stores one document per AI inference cycle containing camera ID, lot ID, slot ID, occupancy flag, confidence score, and frame timestamp with a 30-day TTL index. `slot_history` stores 5-minute aggregated occupancy buckets per lot for dashboard trend charts and future predictive model training. `vehicle_records` will be added in CSE499B for LPR detections and jam event logs, using a flexible document schema to accommodate evolving AI pipeline outputs. MongoDB is written to directly by the FastAPI AI service without routing through NestJS.

C. API Contracts

REST API contracts are defined for all Sprint 3 endpoints following OpenAPI conventions. Contracts cover authentication (POST `/auth/register`, POST `/auth/login`, POST `/auth/refresh`, POST `/auth/logout`), user session management (GET `/auth/sessions`, DELETE `/auth/sessions/:id`), and parking space management (POST `/parking/spaces`, GET `/parking/spaces/:id`, PATCH `/parking/spaces/:id`, DELETE `/parking/spaces/:id`, GET `/parking/nearby`). Internal contracts between NestJS and FastAPI cover camera registration (POST `/cameras/register`) and slot status ingestion. Swagger documentation will be generated via `@nestjs/swagger` during Sprint 3 implementation.

D. UI/UX Wireframes

Wireframes were prepared for all three client interfaces. The driver mobile app covers the map view with nearby parking pins, parking space detail screen, and navigation entry point. The owner registration portal covers the registration and login screens, space creation form with location picker, and camera RTSP connection step with slot mask drawing interface. The authority dashboard covers the city-wide occupancy map and per-space statistics view. Wireframes are stored in the project support directory.

IV. APPLICATION SCAFFOLDING

A. Backend (NestJS)

The NestJS backend is initialized on port 3001 with `ConfigModule` for environment variable management and a global `PrismaModule` providing database access to all feature modules. Prisma 7.8.0 is used with the PostgreSQL adapter and CommonJS output for NestJS compatibility. The multir vulnerability in the NestJS platform-express dependency is mitigated via an npm override pinning multir to 2.2.0, reducing the audit from 4 high severity findings to zero.

B. Frontend (Next.js)

The Next.js 16 frontend is scaffolded with a landing page displaying the CityPulse logo, title, tagline, and entry point buttons linking to `/login` and `/about`. No auth routes are implemented yet — that is Sprint 3 work.

C. Mobile (Flutter)

The Flutter mobile app is scaffolded with a landing screen matching the web frontend: logo, title, tagline, and “Get Started” / “Learn More” buttons. A full Material 3 theme system is implemented supporting both light and dark modes. No routing or API integration yet.

V. KEY DECISIONS

The slot detection mask polygon is stored per slot in the `parking_slots` table as a JSONB field. This follows the finding from the NSU (2022) paper that per-camera slot boundaries must be defined manually — there is no universal model that auto-segments slots from an arbitrary camera position. The AI service will read these coordinates to crop per-slot patches before inference.

Latitude and longitude are stored as plain floats in `parking_spaces` for portability. A PostGIS geography index will be added in a follow-up raw migration to enable the radius-based nearby parking search query without requiring PostGIS at the schema generation step.

The Bangladesh administrative hierarchy (division, district, thana, area) is stored as indexed string columns rather than a normalized reference table. This enables fast zone-level filtering and city-wide analytics by thana without a join, and avoids the maintenance overhead of keeping a reference table synchronized with administrative boundary changes.

VI. PLAN FOR SPRINT 3

Implement JWT authentication with permission-based RBAC, refresh token rotation with multi-session support, and OAuth integration for Google login. Build the owner registration flow including parking space creation with location input, photo upload to S3-compatible storage, and RTSP camera connection. Begin the owner portal UI in Next.js with registration and login screens.

ARTIFACTS

All deliverables from this sprint are available in the project repository.

- Architecture document (PDF): <https://github.com/rahad-dll/499/blob/feature/backend/applications/web/backend/docs/sysArch.pdf>
- ERD image: <https://github.com/rahad-dll/499/blob/feature/backend/applications/web/backend/docs/erd.png>
- Prisma schema: <https://github.com/rahad-dll/499/blob/feature/backend/applications/web/backend/prisma/schema.prisma>
- Initial migration SQL: https://github.com/rahad-dll/499/blob/feature/backend/applications/web/backend/prisma/migrations/20260708010310_init_schema/migration.sql
- NestJS backend source: <https://github.com/rahad-dll/499/tree/feature/backend/applications/web/backend/src>
- Next.js frontend: <https://github.com/rahad-dll/499/tree/development/applications/web/frontend>
- Flutter mobile app: <https://github.com/rahad-dll/499/tree/development/applications/mobile>